

CPRE 431

M06 Lab HW

Assignments will be submitted in PDF format via Canvas.

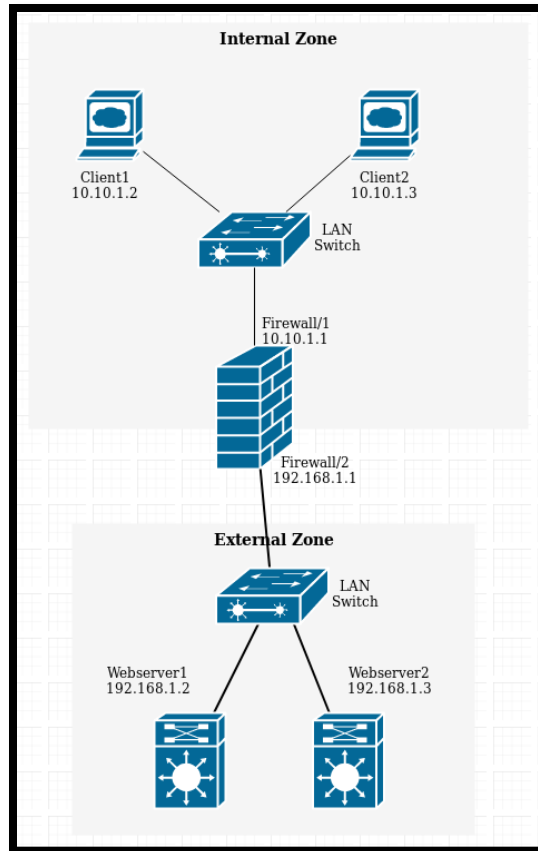
Please submit your homework online through Canvas. Late homework will not be accepted.

Important: Your submission must be in .pdf format ONLY!

Please ensure that you support all your answers with the correct screenshots showing your solutions.

Notes:

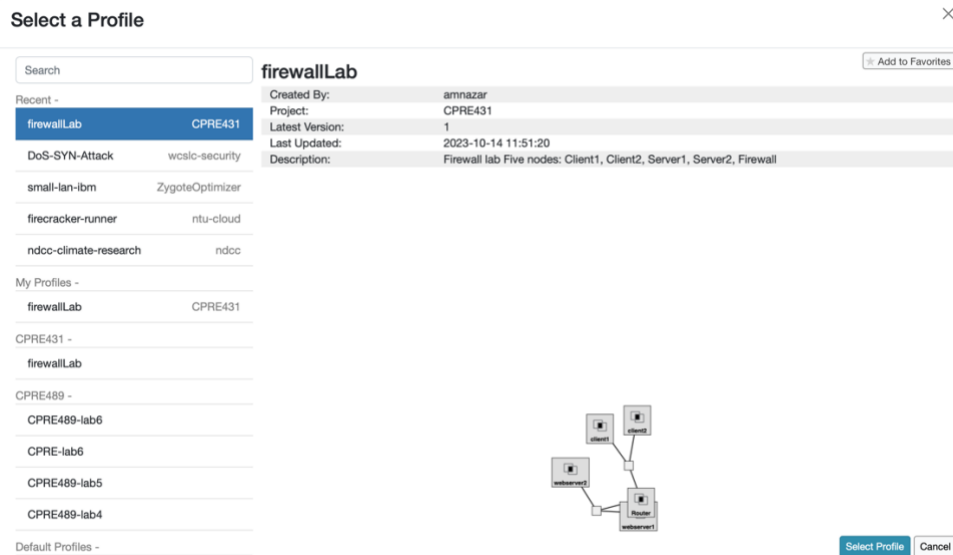
- The goal of this lab is to use iptables to create simple firewall rules.
- Use the “Introduction to iptables” page on canvas to help you in the needed commands for this homework.
- This homework consists of 5 tasks. **The answer for each task requires you to record a command/result and write an explanation and support this with screenshots.**
- Use the template "**M06 - Answers.txt**" file (attached with the homework) as a guideline of how the answer should look like. Your answer submission must be in .pdf format ONLY!
- This virtual network consists of two zones, internal (Clients) and external (Servers). The internal zone consists of two clients (Client1 and 2) connected through a LAN switch to a Firewall that connects them to the external zone. The Firewall has two interfaces “Firewall/1” connected to the internal network, and “Firewall/2” connected to the external zone. The external zone has many servers connected through a LAN switch to the Firewall. However, we will only focus on two servers (Webserver1 and 2). The below figure has the network diagram and the needed details for the zones, clients, servers, and firewall:



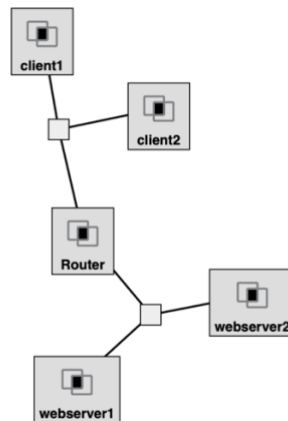
- **In all your answers, try to write rules as general as possible.** For example, although we are only focusing on two servers in the external zone, try to write rules such that the policy is achieved even if there were more than two servers.
- **Always remember to flush all iptables rules between tasks** (for example: after completing task 1, make sure there are no rules before you start task 2).

Before Starting:

1. Login to CloudLab and start a new experiment.
2. When choosing profile, select the **firewallLab** profile as seen below



3. The rest of the instructions should be fairly similar to Lab 4 for initialization.
4. Once your nodes are ready, use the same key pair to ssh into the nodes and complete the lab
5. The expected topology will look very similar to the topology seen below



Task 1: (Aim: understand the difference between INPUT, FORWARD, and OUTPUT chains):

1. Add **Rule1** to the firewall filter table that blocks ping (icmp) packets being forwarded between the two subnets.

The below rule is general because it was not specified which subnet to use, so it will simply block all ping packets going through the router:

```
sudo iptables -A FORWARD -p icmp -j DROP
```

```

[dhiman@router:~$ sudo iptables -A FORWARD -p icmp -j DROP
[dhiman@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
DROP      icmp -- 0.0.0.0/0              0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

2. Delete Rule1, and then add **Rule2** that blocks ping packets coming into the firewall.

The below rule ensures that ping packet traffic coming into the firewall is blocked:

`sudo iptables -A INPUT -p icmp -j DROP`

```

[dhiman@router:~$ sudo iptables -D FORWARD -p icmp -j DROP
[dhiman@router:~$ sudo iptables -A INPUT -p icmp -j DROP
[dhiman@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      icmp -- 0.0.0.0/0              0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

3. Delete Rule2, and then add **Rule3** that blocks ping packets coming out of the firewall.

This rule ensures that ping packets going out of the firewall are blocked:

`sudo iptables -A OUTPUT -p icmp -j DROP`

```

[dhiman@router:~$ sudo iptables -D INPUT -p icmp -j DROP
[dhiman@router:~$ sudo iptables -A OUTPUT -p icmp -j DROP
[dhiman@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
DROP      icmp -- 0.0.0.0/0              0.0.0.0/0

```

To test each of the above rules, try to ping between the following pairs of nodes, and observe whether it is successful or not:

- Client1 to Webserver1
 - Webserver2 to Client2
 - Client1 to Firewall
 - Firewall to Webserver1
 - Client1 to Client2
4. Record your rules' results (allowed or blocked) and an explanation of the difference between the three chains in your answer (take screenshots of webserver/client/firewall terminals to show your syntax and results of pings).

Rule 1:

- **Client1 to Webserver1 (Blocked)**

```
dhiman6@client1:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
^C
--- 10.10.1.3 ping statistics ---
51 packets transmitted, 0 received, 100% packet loss, time 51190ms
```

The above did not work because Rule 1 blocks ping packets being forwarded between the two subnets.

```
dhiman6@router:~$ sudo iptables -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
107 8988 DROP icmp -- * * 0.0.0.0/0 0.0.0.0/0
```

The above screenshot shows on the router the number of packets that were dropped by the FORWARD chain.

- **Webserver2 to Client2 (Blocked)**

```
dhiman6@webserver2:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
^C
--- 10.10.2.3 ping statistics ---
6 packets transmitted, 0 received, 100% packet loss, time 5112ms
```

The above shows Webserver2 was not able to ping because again Rule 1 blocks ping packets from being transmitted between the two subnets.

```
dhiman6@router:~$ sudo iptables -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
5 420 DROP icmp -- * * 0.0.0.0/0 0.0.0.0/0
```

The above screenshot shows on the router the number of packets that were dropped by the FORWARD chain.

- **Client1 to Firewall (Allowed)**

```
dhiman6@client1:~$ ping 10.10.2.1
PING 10.10.2.1 (10.10.2.1) 56(84) bytes of data.
64 bytes from 10.10.2.1: icmp_seq=1 ttl=64 time=0.585 ms
64 bytes from 10.10.2.1: icmp_seq=2 ttl=64 time=0.261 ms
64 bytes from 10.10.2.1: icmp_seq=3 ttl=64 time=0.339 ms
64 bytes from 10.10.2.1: icmp_seq=4 ttl=64 time=0.372 ms
64 bytes from 10.10.2.1: icmp_seq=5 ttl=64 time=0.371 ms
64 bytes from 10.10.2.1: icmp_seq=6 ttl=64 time=0.165 ms
64 bytes from 10.10.2.1: icmp_seq=7 ttl=64 time=0.340 ms
64 bytes from 10.10.2.1: icmp_seq=8 ttl=64 time=0.257 ms
^C
--- 10.10.2.1 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7154ms
rtt min/avg/max/mdev = 0.165/0.336/0.585/0.114 ms
```

The above shows that Client1 is able to ping the Firewall. This is because Rule 1 only blocks packets being forwarded. If we were using Rule 2 for INPUT then it would not have worked.

- **Firewall to Webserver1 (Allowed)**

```
dhiman6@router:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
64 bytes from 10.10.1.3: icmp_seq=1 ttl=64 time=0.763 ms
64 bytes from 10.10.1.3: icmp_seq=2 ttl=64 time=0.271 ms
64 bytes from 10.10.1.3: icmp_seq=3 ttl=64 time=0.253 ms
64 bytes from 10.10.1.3: icmp_seq=4 ttl=64 time=0.413 ms
^C
--- 10.10.1.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3054ms
rtt min/avg/max/mdev = 0.253/0.425/0.763/0.204 ms
```

Again, we are able to ping from Firewall to Webserver1 because our Rule 1 is for forwarded traffic not outgoing traffic.

- **Client1 to Client2 (Allowed)**

```
dhiman6@client1:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
64 bytes from 10.10.2.3: icmp_seq=1 ttl=64 time=0.912 ms
64 bytes from 10.10.2.3: icmp_seq=2 ttl=64 time=0.481 ms
64 bytes from 10.10.2.3: icmp_seq=3 ttl=64 time=0.489 ms
64 bytes from 10.10.2.3: icmp_seq=4 ttl=64 time=0.490 ms
^C
--- 10.10.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.481/0.593/0.912/0.184 ms
```

We are able to ping from Client1 to Client2 because we are not going through the router and are pinging from one device to another.

Rule 2:

- **Client1 to Webserver1 (Allowed)**

```
dhiman6@client1:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
64 bytes from 10.10.1.3: icmp_seq=1 ttl=63 time=1.09 ms
64 bytes from 10.10.1.3: icmp_seq=2 ttl=63 time=0.604 ms
64 bytes from 10.10.1.3: icmp_seq=3 ttl=63 time=0.625 ms
64 bytes from 10.10.1.3: icmp_seq=4 ttl=63 time=0.698 ms
^C
--- 10.10.1.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3052ms
rtt min/avg/max/mdev = 0.604/0.754/1.091/0.197 ms
```

This is allowed since the ping packets are not being sent to the router/firewall but instead are being forwarded from Client1 to Webserver1.

- **Webserver2 to Client2 (Allowed)**

```
dhiman6@webserver2:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
64 bytes from 10.10.2.3: icmp_seq=1 ttl=63 time=1.21 ms
64 bytes from 10.10.2.3: icmp_seq=2 ttl=63 time=0.625 ms
64 bytes from 10.10.2.3: icmp_seq=3 ttl=63 time=0.684 ms
64 bytes from 10.10.2.3: icmp_seq=4 ttl=63 time=0.894 ms
^C
--- 10.10.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.625/0.854/1.214/0.230 ms
```

The above is also allowed since the ping packets are destined for Client2 and not the router/firewall.

- **Client1 to Firewall (Blocked)**

```
dhiman6@client1:~$ ping 10.10.2.1
PING 10.10.2.1 (10.10.2.1) 56(84) bytes of data.
^C
--- 10.10.2.1 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3074ms
```

The above is blocked because the ping packets are being sent directly to the firewall which is caught by Rule 2 and dropped.

- **Firewall to Webserver1 (Blocked)**

```
dhiman6@router:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
^C
--- 10.10.1.3 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3059ms
```

The above is also blocked because we are allowed to send the ping to the Webserver1 but when it replies, that reply comes into the Firewall's INPUT chain and gets dropped. This is why it looks as though we are unable to ping Webserver1, in reality we may/may not be pinging the device, we just are not receiving any replies because they are being dropped.

- **Client1 to Client2 (Allowed)**

```
dhiman6@client1:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
64 bytes from 10.10.2.3: icmp_seq=1 ttl=64 time=0.509 ms
64 bytes from 10.10.2.3: icmp_seq=2 ttl=64 time=0.457 ms
64 bytes from 10.10.2.3: icmp_seq=3 ttl=64 time=0.479 ms
^C
--- 10.10.2.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2038ms
rtt min/avg/max/mdev = 0.457/0.481/0.509/0.021 ms
```

We can ping from Client1 to Client2 because again Rule2 does not block forwarded packets and only cares about packets coming into the Firewall that are ping which it drops.

Rule 3:

- **Client1 to Webserver1 (Allowed)**

```
dhiman6@client1:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
64 bytes from 10.10.1.3: icmp_seq=1 ttl=63 time=0.973 ms
64 bytes from 10.10.1.3: icmp_seq=2 ttl=63 time=1.01 ms
64 bytes from 10.10.1.3: icmp_seq=3 ttl=63 time=0.999 ms
64 bytes from 10.10.1.3: icmp_seq=4 ttl=63 time=0.999 ms
^C
--- 10.10.1.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.973/0.995/1.012/0.014 ms
```

We are able to ping from Client1 to Webserver1 because the packet is being forwarded and Rule3 only cares about the OUTPUTs being sent from the router/firewall.

- **Webserver2 to Client2 (Allowed)**

```
dhiman6@webserver2:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
64 bytes from 10.10.2.3: icmp_seq=1 ttl=63 time=0.911 ms
64 bytes from 10.10.2.3: icmp_seq=2 ttl=63 time=0.809 ms
64 bytes from 10.10.2.3: icmp_seq=3 ttl=63 time=0.960 ms
64 bytes from 10.10.2.3: icmp_seq=4 ttl=63 time=1.05 ms
^C
--- 10.10.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.809/0.933/1.054/0.088 ms
```

We are able to ping from Webserver2 to Client2 for the same reason as the above. The packets are being forwarded so the OUTPUT chain on the router/firewall is not involved.

- **Client1 to Firewall (Blocked)**

```
dhiman6@client1:~$ ping 10.10.2.1
PING 10.10.2.1 (10.10.2.1) 56(84) bytes of data.
^C
--- 10.10.2.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2045ms
```

In this case our Rule3 kicks in because now the ping packets are being sent to the firewall, so they are received, but then on the response from the firewall to Client1 they are blocked because Rule3 drops all OUTPUT packets from the Firewall.

- **Firewall to Webserver1 (Blocked)**

```
dhiman6@router:~$ ping 10.10.1.3
PING 10.10.1.3 (10.10.1.3) 56(84) bytes of data.
^C
--- 10.10.1.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2045ms
```

In this case our Rule3 is still involved because the Firewall is trying to send ping packets to Webserver1 so the OUTPUT chain kicks in. They are not being forwarded but instead they originate from the Firewall so they are blocked by Rule3.

- **Client1 to Client2 (Allowed)**

```
dhiman6@client1:~$ ping 10.10.2.3
PING 10.10.2.3 (10.10.2.3) 56(84) bytes of data.
64 bytes from 10.10.2.3: icmp_seq=1 ttl=64 time=0.584 ms
64 bytes from 10.10.2.3: icmp_seq=2 ttl=64 time=0.479 ms
64 bytes from 10.10.2.3: icmp_seq=3 ttl=64 time=0.329 ms
64 bytes from 10.10.2.3: icmp_seq=4 ttl=64 time=0.481 ms
^C
--- 10.10.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3068ms
rtt min/avg/max/mdev = 0.329/0.468/0.584/0.090 ms
```

Since we are forwarding the packet from Client1 to Client2 it is not blocked by Rule3.

In all the above cases I did not really show the clients because if packets are blocked then the client won't get them anyway and I wasn't sure what to show. In the case where the client is receiving the packets then you can see that from the senders side which I have provided.

Task 2: (Aim: filter based on ports and IPs):

Before Rule4:

```
dhiman6@client1:~$ ssh test@webserver1
test@webserver1's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

https://ubuntu.com/pro
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Nov  2 16:52:22 2025 from 10.10.2.2
test@webserver1:~$ exit
logout
Connection to webserver1 closed.
```

Below is how I set Rule4 to block Client1 from SSHing to any outside nodes but also be able to SSH to the firewall itself. I had to ping Router from client1 to find out its IP addr, then I set the rules below:

sudo iptables -A OUTPUT -p tcp -d 10.10.2.1 --dport 22 -j ACCEPT

sudo iptables -A OUTPUT -p tcp --dport 22 -j DROP

```
dhiman6@client1:~$ sudo iptables -A OUTPUT -p tcp -d 10.10.2.1 --dport 22 -j ACCEPT
dhiman6@client1:~$ sudo iptables -A OUTPUT -p tcp --dport 22 -j DROP
dhiman6@client1:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  0.0.0.0/0            10.10.2.1            tcp dpt:22
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:22

dhiman6@client1:~$ ssh test@Router
test@router's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

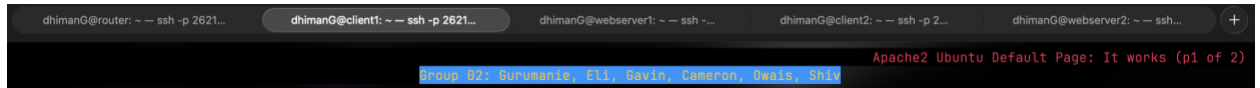
https://ubuntu.com/pro
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Nov  2 17:21:00 2025 from 10.10.2.2
test@router:~$ exit
logout
Connection to router closed.
dhiman6@client1:~$ ssh test@webserver1 -o ConnectTimeout=5
ssh: connect to host webserver1 port 22: Connection timed out
dhiman6@client1:~$ ssh test@webserver2 -o ConnectTimeout=5
ssh: connect to host webserver2 port 22: Connection timed out
```

As we can see from the above, we are able to ssh into Router but nothing else.

Task 3: (Aim: filter based on ports, understand HTTP request/response format):

Client1 connected to Webserver1 with group number and member names:



Client2 connected to Webserver2 with group number and member names:



The rule I added was to simply stop all forwarded traffic using tcp protocol on port 80, while specifying the destination as 10.10.1.3 which is Webserver1. If I did not specify the destination then both internal devices would be unable to access both Webservers.

```
dhimanG@router:~$ sudo iptables -A FORWARD -p tcp -d 10.10.1.3 --dport 80 -j DROP
dhimanG@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
DROP      tcp  --  0.0.0.0/0             10.10.1.3             tcp dpt:80

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

Now both clients 1 and 2 show the below when trying to connect to Webserver1 but are both able to access Webserver2.

```
Making HTTP connection to webserver1
dhimanG@client1:~$ lynx http://webserver1
Looking up webserver1
Making HTTP connection to webserver1
Alert!: Unable to connect to remote host.
lynx: Can't access startfile http://webserver1/
dhimanG@client2:~$ lynx http://webserver1
Looking up webserver1
Making HTTP connection to webserver1
Alert!: Unable to connect to remote host.
lynx: Can't access startfile http://webserver1/
```

Task 4: (Aim: change default policy):

Looking into the man page for iptables, it says we need to use -P flag to specify policy for a specific chain, after this I ran the following command:

`sudo iptables -P FORWARD DROP`

```
dhiman@router:~$ sudo iptables -P FORWARD DROP
dhiman@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Verification of parts 1 to 3 after default policy of DROP is applied for forwarding:

Client1:

```
dhiman@client1:~$ ping webserver1
PING webserver1-link-0 (10.10.1.3) 56(84) bytes of data.
^C
--- webserver1-link-0 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2031ms

dhiman@client1:~$ ping webserver2
PING webserver2-link-0 (10.10.1.1) 56(84) bytes of data.
^C
--- webserver2-link-0 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1026ms

dhiman@client1:~$ ping Router
PING Router-link-1 (10.10.2.1) 56(84) bytes of data.
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=1 ttl=64 time=0.571 ms
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=2 ttl=64 time=0.483 ms
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=3 ttl=64 time=0.383 ms
^C
--- Router-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2050ms
rtt min/avg/max/mdev = 0.383/0.479/0.571/0.076 ms

dhiman@client1:~$ ping client2
PING client2-link-1 (10.10.2.3) 56(84) bytes of data.
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=1 ttl=64 time=0.508 ms
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=2 ttl=64 time=0.483 ms
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=3 ttl=64 time=0.493 ms
^C
--- client2-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.483/0.494/0.508/0.010 ms
```

Webserver1:

```
dhiman@webserver1:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
^C
--- client1-link-1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2049ms

dhiman@webserver1:~$ ping client2
PING client2-link-1 (10.10.2.3) 56(84) bytes of data.
^C
--- client2-link-1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2051ms

dhiman@webserver1:~$ ping webserver2
PING webserver2-link-0 (10.10.1.1) 56(84) bytes of data.
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=1 ttl=64 time=0.819 ms
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=2 ttl=64 time=0.475 ms
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=3 ttl=64 time=0.511 ms
^C
--- webserver2-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.475/0.601/0.819/0.154 ms

dhiman@webserver1:~$ ping Router
PING Router-link-0 (10.10.1.2) 56(84) bytes of data.
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=1 ttl=64 time=0.431 ms
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=2 ttl=64 time=0.482 ms
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=3 ttl=64 time=0.481 ms
^C
--- Router-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.431/0.464/0.482/0.023 ms
```

Client2:

```
dhiman@client2:~$ ping webserver1
PING webserver1-link-0 (10.10.1.3) 56(84) bytes of data.
^C
--- webserver1-link-0 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1005ms

dhiman@client2:~$ ping webserver2
PING webserver2-link-0 (10.10.1.1) 56(84) bytes of data.
^C
--- webserver2-link-0 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2048ms

dhiman@client2:~$ ping Router
PING Router-link-1 (10.10.2.1) 56(84) bytes of data.
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=1 ttl=64 time=0.435 ms
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=2 ttl=64 time=0.433 ms
64 bytes from Router-link-1 (10.10.2.1): icmp_seq=3 ttl=64 time=0.430 ms
^C
--- Router-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2037ms
rtt min/avg/max/mdev = 0.430/0.432/0.435/0.002 ms
dhiman@client2:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=1 ttl=64 time=0.430 ms
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=2 ttl=64 time=0.393 ms
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=3 ttl=64 time=0.393 ms
^C
--- client1-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2031ms
rtt min/avg/max/mdev = 0.393/0.405/0.430/0.017 ms
```

Webserver2:

```
dhiman@webserver2:~$ ping webserver1
PING webserver1-link-0 (10.10.1.3) 56(84) bytes of data.
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=1 ttl=64 time=0.584 ms
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=2 ttl=64 time=0.497 ms
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=3 ttl=64 time=0.503 ms
^C
--- webserver1-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.497/0.528/0.584/0.039 ms
dhiman@webserver2:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
^C
--- client1-link-1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2048ms

dhiman@webserver2:~$ ping client2
PING client2-link-1 (10.10.2.3) 56(84) bytes of data.
^C
--- client2-link-1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2026ms

dhiman@webserver2:~$ ping Router
PING Router-link-0 (10.10.1.2) 56(84) bytes of data.
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=1 ttl=64 time=0.496 ms
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=2 ttl=64 time=0.311 ms
64 bytes from Router-link-0 (10.10.1.2): icmp_seq=3 ttl=64 time=0.520 ms
^C
--- Router-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 0.311/0.442/0.520/0.093 ms
```

Router:

```
dhiman@router:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=1 ttl=64 time=0.567 ms
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=2 ttl=64 time=0.475 ms
64 bytes from client1-link-1 (10.10.2.2): icmp_seq=3 ttl=64 time=0.336 ms
^C
--- client1-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2015ms
rtt min/avg/max/mdev = 0.336/0.459/0.567/0.094 ms
dhiman@router:~$ ping client2
PING client2-link-1 (10.10.2.3) 56(84) bytes of data.
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=1 ttl=64 time=0.605 ms
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=2 ttl=64 time=0.475 ms
64 bytes from client2-link-1 (10.10.2.3): icmp_seq=3 ttl=64 time=0.483 ms
^C
--- client2-link-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 0.475/0.521/0.605/0.059 ms
dhiman@router:~$ ping webserver1
PING webserver1-link-0 (10.10.1.3) 56(84) bytes of data.
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=1 ttl=64 time=0.625 ms
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=2 ttl=64 time=0.364 ms
64 bytes from webserver1-link-0 (10.10.1.3): icmp_seq=3 ttl=64 time=0.417 ms
^C
--- webserver1-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2022ms
rtt min/avg/max/mdev = 0.364/0.468/0.625/0.112 ms
dhiman@router:~$ ping webserver2
PING webserver2-link-0 (10.10.1.1) 56(84) bytes of data.
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=1 ttl=64 time=0.537 ms
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=2 ttl=64 time=0.578 ms
64 bytes from webserver2-link-0 (10.10.1.1): icmp_seq=3 ttl=64 time=0.530 ms
^C
--- webserver2-link-0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.530/0.548/0.578/0.021 ms
```


Task 5: (Aim: Use Stateful Packet Inspection):

Enabling Stateful Packet Inspection:

```
dhiman6@router:~$ sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
dhiman6@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target    prot opt source                destination

Chain FORWARD (policy DROP)
target    prot opt source                destination
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0              state RELATED,ESTABLISHED

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
```

For rule 6 I used -i flag for eth1 and -o flag for eth0. Apart from that the rest we have done in the previous parts as well where we set destination port to 80 and specify the jump to be ACCEPT.

sudo iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 80 -j ACCEPT

```
dhiman6@router:~$ sudo iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 80 -j ACCEPT
dhiman6@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target    prot opt source                destination

Chain FORWARD (policy DROP)
target    prot opt source                destination
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0              state RELATED,ESTABLISHED
ACCEPT    tcp  --  0.0.0.0/0              0.0.0.0/0              tcp dpt:80

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
```

With rule 6 we are able to test it to see that Client2 is unable to connect to Webserver1:

```
dhiman6@client2:~$ lynx http://webserver1

Looking up webserver1
Making HTTP connection to webserver1
Alert!: Unable to connect to remote host.

lynx: Can't access startfile http://webserver1/
```

For rule 7, I just specified the input interface as eth0 and the output as eth1, then the rest is the usual where we specify the protocol as tcp (since ssh uses tcp) and the destination to be the IP addr for the Client1, and the jump is to ACCEPT:

sudo iptables -A FORWARD -p tcp --dport 22 -d 10.10.2.2 -j ACCEPT

```
dhiman6@router:~$ sudo iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 22 -d 10.10.2.2 -j ACCEPT
dhiman6@router:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target    prot opt source                destination

Chain FORWARD (policy DROP)
target    prot opt source                destination
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0              state RELATED,ESTABLISHED
ACCEPT    tcp  --  0.0.0.0/0              0.0.0.0/0              tcp dpt:80
ACCEPT    tcp  --  0.0.0.0/0              10.10.2.2              tcp dpt:22

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
```

With rule 7 we are able to ssh into client1 from Webserver1.

```
dhiman6@webserver1:~$ ssh test@client1
test@client1's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

   https://ubuntu.com/pro
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

test@client1:~$ exit
logout
Connection to client1 closed.
```

To test all other connections to ensure that they are blocked

Unable to ping client1 from webserver1 but we were able to ssh:

```
[dhiman6@webserver1:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
^C
--- client1-link-1 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1001ms
```

Able to ssh from webserver2 into client1:

```
dhiman6@webserver2:~$ ssh test@client1
The authenticity of host 'client1 (10.10.2.2)' can't be established.
ED25519 key fingerprint is SHA256:gkFN/SvBkCeQfIkDSYilK7WgB0UyfXwZPIEyp0Q9hYY.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'client1' (ED25519) to the list of known hosts.
test@client1's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

   https://ubuntu.com/pro
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Nov  2 18:52:55 2025 from 10.10.1.3
test@client1:~$ exit
logout
Connection to client1 closed.
```

Unable to ping client1 from webserver2:

```
dhimanG@webserver2:~$ ping client1
PING client1-link-1 (10.10.2.2) 56(84) bytes of data.
^C
--- client1-link-1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2044ms
```

Able to ssh from router to client1:

```
dhimanG@router:~$ ssh test@client1
The authenticity of host 'client1 (10.10.2.2)' can't be established.
ED25519 key fingerprint is SHA256:qkFN/SvBkCeQfIkD5YilK7WgB0UyfXwZPIEyp0Q9hYY.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'client1' (ED25519) to the list of known hosts.
test@client1's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

https://ubuntu.com/pro
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Nov  2 19:00:49 2025 from 10.10.1.3
test@client1:~$ exit
logout
Connection to client1 closed.
```